Name: Y.Veerendar Reddy **Reg.No:-** 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT – 4

Exploration vs. Exploitation Analysis Using Multi-Armed Bandit Algorithms

Aim

To implement a Multi-Armed Bandit algorithm and analyze the trade-off between exploration and exploitation while selecting the best-performing arm.

Procedure

1. Create multiple arms with different reward probabilities.
2. Initialize the estimated reward for each arm.
3. Use the ε-greedy algorithm to select an arm.
4. Perform exploration with probability ε.
5. Perform exploitation by selecting the arm with the highest estimated reward.
6. Generate rewards for each selected arm.
7. Update the estimated reward of the selected arm.
8. Repeat the process for multiple trials.
9. Calculate the total reward and average reward.
10. Visualize arm selection frequency and cumulative reward.

Code:

```
# ============================================================
# EXPERIMENT 4
# Exploration vs. Exploitation Analysis
# Using Multi-Armed Bandit Algorithm
# ============================================================

import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
```

```
# STEP 1: Set Random Seed
# ------------------------------------------------------------

np.random.seed(42)

# ------------------------------------------------------------
# STEP 2: Define Multi-Armed Bandit
# ------------------------------------------------------------

# True probability of receiving reward = 1
# for each arm

true_probabilities = [0.20, 0.50, 0.70, 0.40, 0.60]

num_arms = len(true_probabilities)

# Number of trials
num_trials = 1000

# Exploration probability
epsilon = 0.10

# ------------------------------------------------------------
# STEP 3: Initialize Variables
# ------------------------------------------------------------

# Estimated reward for each arm
estimated_rewards = np.zeros(num_arms)

# Number of times each arm is selected
arm_counts = np.zeros(num_arms)
```

```
# Store selected arms
selected_arms = []

# Store rewards
rewards = []

# Store cumulative rewards
cumulative_rewards = []

total_reward = 0

# ------------------------------------------------------------
# STEP 4: Epsilon-Greedy Algorithm
# ------------------------------------------------------------

for trial in range(num_trials):

    # --------------------------------------------------------
    # Exploration
    # --------------------------------------------------------

    if np.random.random() < epsilon:

        # Select a random arm
        arm = np.random.randint(num_arms)

    # --------------------------------------------------------
    # Exploitation
    # --------------------------------------------------------
```

```
else:

    # Select arm having highest estimated reward
    arm = np.argmax(estimated_rewards)

# -----------------------------------------------------

# Generate Reward

# -----------------------------------------------------

reward = 1 if np.random.random() < true_probabilities[arm] else 0

# -----------------------------------------------------

# Update Arm Information

# -----------------------------------------------------

arm_counts[arm] += 1

# Incremental average reward calculation
estimated_rewards[arm] += (
    reward - estimated_rewards[arm]
) / arm_counts[arm]

# -----------------------------------------------------

# Store Results

# -----------------------------------------------------

selected_arms.append(arm)
rewards.append(reward)

total_reward += reward
```

```
    cumulative_rewards.append(total_reward)



# ------------------------------------------------------------
# STEP 5: Display Results
# ------------------------------------------------------------

print("=" * 60)
print("MULTI-ARMED BANDIT RESULTS")
print("=" * 60)

print("\nTrue Reward Probabilities:")

for i in range(num_arms):
    print(
        f"Arm {i + 1}: "
        f"{true_probabilities[i]:.2f}"
    )

print("\nEstimated Reward Probabilities:")

for i in range(num_arms):
    print(
        f"Arm {i + 1}: "
        f"{estimated_rewards[i]:.4f}"
    )

print("\nNumber of Times Each Arm Was Selected:")

for i in range(num_arms):
    print(
```

```
        f"Arm {i + 1}: "
        f"{int(arm_counts[i])}"
    )

print("\nBest Arm Based on Estimated Reward:",
      np.argmax(estimated_rewards) + 1)

print("Total Reward:", total_reward)

print("Average Reward:",
      total_reward / num_trials)



# ---------------------------------------------------------
# STEP 6: Arm Selection Frequency Visualization
# ---------------------------------------------------------

plt.figure(figsize=(9, 5))

plt.bar(
    range(1, num_arms + 1),
    arm_counts
)

plt.title("Arm Selection Frequency")
plt.xlabel("Arm")
plt.ylabel("Number of Selections")

plt.xticks(range(1, num_arms + 1))

plt.grid(axis="y")
```

```
plt.show()


# ------------------------------------------------------------
# STEP 7: Estimated Reward Visualization
# ------------------------------------------------------------

plt.figure(figsize=(9, 5))

plt.bar(
    range(1, num_arms + 1),
    estimated_rewards
)

plt.title("Estimated Reward of Each Arm")
plt.xlabel("Arm")
plt.ylabel("Estimated Reward Probability")

plt.xticks(range(1, num_arms + 1))

plt.ylim(0, 1)

plt.grid(axis="y")

plt.show()


# ------------------------------------------------------------
# STEP 8: Cumulative Reward Visualization
# ------------------------------------------------------------
```

```
plt.figure(figsize=(10, 5))

plt.plot(
    range(1, num_trials + 1),
    cumulative_rewards
)

plt.title("Cumulative Reward Over Trials")
plt.xlabel("Trial")
plt.ylabel("Cumulative Reward")

plt.grid(True)

plt.show()



# ---------------------------------------------------------
# STEP 9: Exploration vs Exploitation Analysis
# ---------------------------------------------------------

exploration_count = sum(
    1 for i in range(num_trials)
    if i > 0 and selected_arms[i] != np.argmax(estimated_rewards)
)

exploitation_count = num_trials - exploration_count

print("\n" + "=" * 60)
print("EXPLORATION VS EXPLOITATION")
print("=" * 60)
```

```
print("Total Trials:", num_trials)
print("Exploration Probability:", epsilon)
print("Approximate Exploration Trials:",
    int(num_trials * epsilon))
print("Approximate Exploitation Trials:",
    num_trials - int(num_trials * epsilon))
```

OUTPUT:-

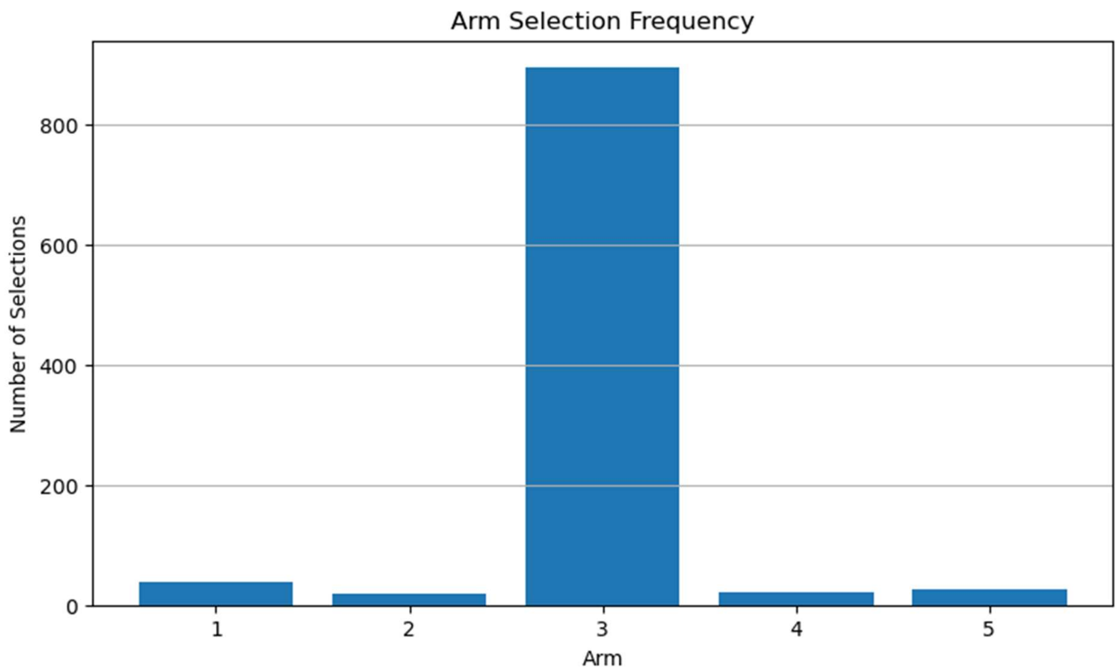
```
============================================================
MULTI-ARMED BANDIT RESULTS
============================================================

True Reward Probabilities:
Arm 1: 0.20
Arm 2: 0.50
Arm 3: 0.70
Arm 4: 0.40
Arm 5: 0.60

Estimated Reward Probabilities:
Arm 1: 0.1842
Arm 2: 0.4000
Arm 3: 0.7148
Arm 4: 0.4091
Arm 5: 0.6154

Number of Times Each Arm Was Selected:
Arm 1: 38
Arm 2: 20
Arm 3: 894
Arm 4: 22
```

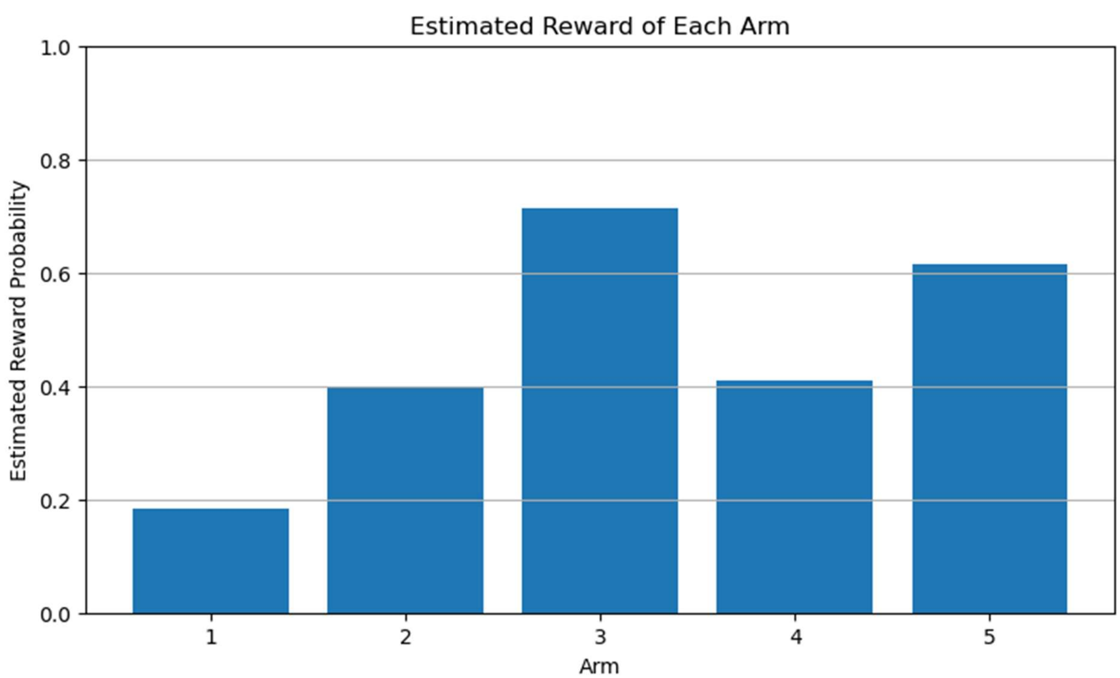
Arm 5: 26

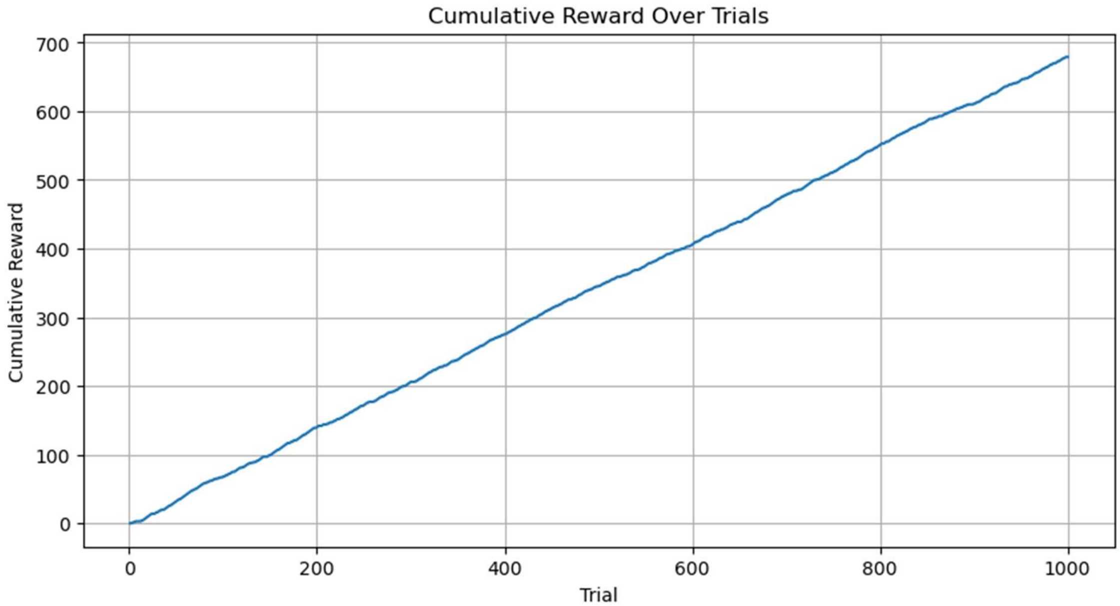
Best Arm Based on Estimated Reward: 3

Total Reward: 679

Average Reward: 0.679







==

EXPLORATION VS EXPLOITATION

==

Total Trials: 1000

Exploration Probability: 0.1

Approximate Exploration Trials: 100

Approximate Exploitation Trials: 900

Visualization

The program generates three important graphs:

1. Arm Selection Frequency

Shows how many times each arm was selected.

The best-performing arm should generally receive more selections because the ε-greedy algorithm increasingly exploits the arm with the highest estimated reward.

2. Estimated Reward of Each Arm

Shows the estimated probability of receiving a reward from each arm.

The algorithm should gradually identify the arm having the highest reward probability.

3. Cumulative Reward

Shows the total reward accumulated as the number of trials increases.

A steadily increasing curve indicates that the algorithm is successfully collecting rewards.

Summary

The ε-greedy Multi-Armed Bandit algorithm was implemented to study the balance between exploration and exploitation. The algorithm explores different arms initially and gradually exploits the arm with the highest estimated reward.

Result

Thus, the exploration–exploitation trade-off was successfully analyzed using the Multi-Armed Bandit algorithm, and arm selection, estimated rewards, and cumulative rewards were visualized.